Maze Search: Greedy Best-First vs A*

A C++ program that solves a 5x5 maze using two informed search algorithms, **Greedy Best-First Search** and **A***, each with two heuristics, and compares their cost and number of expanded nodes from 5 different start states.

Maze Layout

```
....X
#~###
.~...
#~#.#
.....
```

Symbol	Meaning
`.`	Empty cell (walkable)
`~`	Flooded cell (walkable)
`G`	Walkable cell
`X`	Goal
`#`	Wall (blocked)
`A`	Start position (in printed output)
`*`	Cells on the solution path (in printed output)

Features

- **Two algorithms:**
  - Greedy Best-First Search: `f(n) = h(n)`
  - A* Search: `f(n) = g(n) + h(n)`
- **Two heuristics:**
  - `h1` : Euclidean distance to the goal

    - `h2` : Manhattan distance to the goal

- **5 actions:** Up, Down, Left, Right, Stay (each costs 1)
- **5 start states:** `(4,0)` , `(4,4)` , `(2,2)` , `(0,0)` , `(2,0)`
- Prints a comparison table (cost and expanded nodes) and draws the A* (Manhattan) path on the maze for each start state.

How It Works

1. The maze is read and the goal `X` is located.
2. For each start state, the search runs with every combination of algorithm and heuristic.
3. The open list is scanned for the node with the lowest `f` value, and a visited array prevents re-expanding cells.
4. When the goal is reached, the path is rebuilt through parent indices and the total cost is returned. If no path exists, the cost is `-1` .

Build and Run

Requires a C++ compiler (g++, clang++, or MSVC).

```
g++ -o maze main.cpp
./maze
```

On Windows:

```
g++ -o maze.exe main.cpp
maze.exe
```

Output Format

```
Start | Algorithm       | Heuristic | Cost | Expanded
-------------------------------------------------
(4,0) | Greedy          | h1 Euclid | ...  | ...
(4,0) | Greedy          | h2 Manhat.| ...  | ...
(4,0) | A*              | h1 Euclid | ...  | ...
(4,0) | A*              | h2 Manhat.| ...  | ...
   <maze with path drawn>
-------------------------------------------------
```

A cost of `-1` means the goal is unreachable from that start state.

Project Structure

```
.
├── main.cpp    # Maze class, search algorithms, and main()
└── README.md
```

Notes

- Greedy search is faster but does not guarantee the optimal path.
- A* with an admissible heuristic (both h1 and h2 are admissible here) returns the optimal cost.
- Since "Stay" is allowed, the visited array ensures it never causes infinite loops.